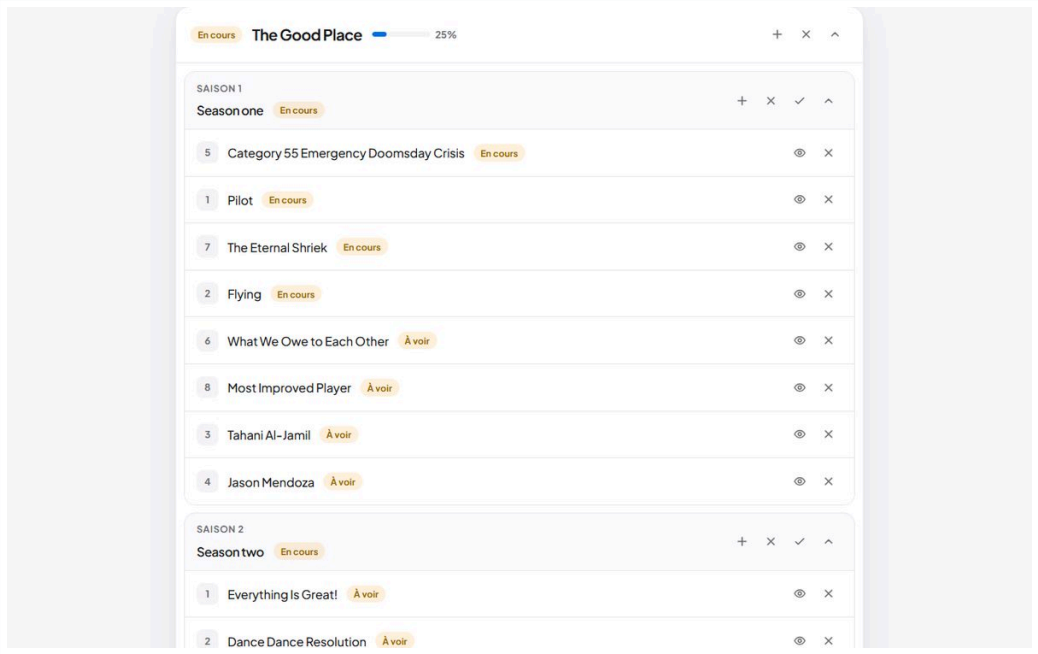
Compte-rendu GestionSeries

TP JEE (sujet AEN-Intech). Stack : Tomcat 10, Jakarta EE, EclipseLink (JPA), MySQL WAMP, Eclipse + Maven. Suivi local de séries TV.

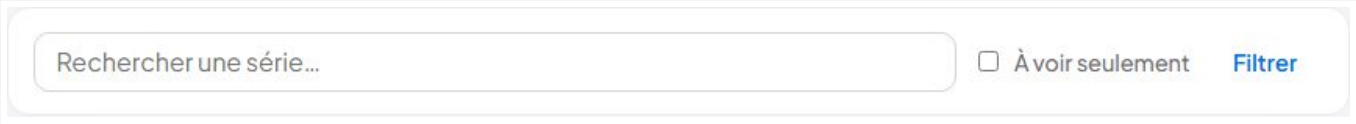
http://localhost:8080/GestionSeries/

Capture · page d·accueil



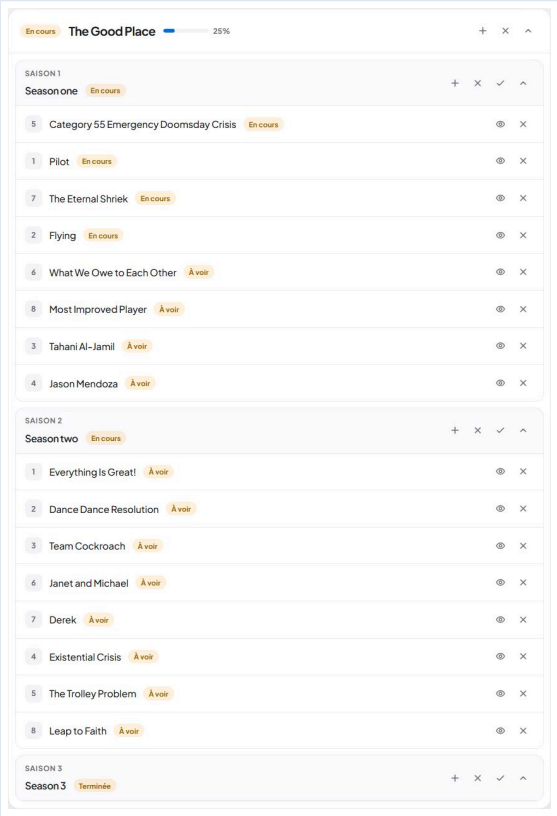
Capture réelle localhost · séries, progression, saisons et épisodes.

Capture · recherche / filtre



Barre recherche + case « À voir seulement » + Filtrer.

Capture · détail d·une série



Progression 25 %, pastilles d·état, icône ·il = bascule « vu ».

Glossaire des termes

Tous les mots techniques du projet, expliqués simplement.

JEE / Jakarta EE	Tech Java pour apps web serveur. Tomcat 10 · packages jakarta.*.
Tomcat	Serveur qui exécute le Java et répond au navigateur (port 8080).
Servlet	Classe liée à une URL (/liste, /etat). Traite la requête puis forward/redirect.
@WebServlet	Annotation qui mappe une URL sur une servlet.
JSP	Page HTML dynamique. Affiche les données de la servlet (${serie.nom}).
JSTL	Balises JSP : <c:forEach>, <c:if>, <c:out>·
GET / POST	GET = lire. POST = modifier (créer, supprimer, marquer vu).
Redirect	Le serveur renvoie le navigateur vers une autre URL.
Forward	Enchaînement interne vers une JSP sans changer l·URL.
DAO	Data Access Object : isole l·accès BDD derrière des interfaces.
Interface	Contrat de méthodes. Ex. ISerieDao · SerieDaoBdd.
CRUD	Create, Read, Update, Delete.
Singleton	Une seule instance (DaoFactory.getInstance()).
Factory	Fabrique qui fournit d·autres objets (getSerieDao()).
JPA	Mappe objets Java · tables SQL (@Entity, @Id·).
EclipseLink	Implémentation JPA utilisée ici.
Entité	Classe @Entity = une table (Serie, Saison, Episode).
persistence.xml	Config JPA : URL MySQL, driver, entités.
EntityManager	Objet JPA qui lit/écrit (persist, merge, remove).
EMF	Usine d·EntityManager, créée une fois au démarrage.
Thread-safe	Correct avec plusieurs requêtes en parallèle.
Filter	Code autour de chaque requête (ouvre/ferme l·EM).
JPQL	Requêtes sur le modèle objet (pas du SQL brut).
Named Query	Requête JPQL nommée, appelée depuis le DAO.
Transaction	Tout validé (commit) ou tout annulé (rollback).
EAGER / LAZY	Charger les relations tout de suite ou à la demande.
JOIN FETCH	Charge les relations jointes dans une requête JPQL.
Maven / pom.xml	Gère les dépendances (JSTL, MySQL, EclipseLink).
Flash message	Message en session affiché une fois après redirect.
ContextPath	Préfixe d·URL (/GestionSeries) pour redirects portables.

1. Conformément au PDF

- p.2·7 Tomcat + import + Run on Server
- p.14·22 DAO interfaces, DaoFactory Singleton + Factory
- p.23 ListeSeries via DaoFactory
- p.24·27 Maven + driver BDD
- p.32·65 JPA, entités, named queries, transactions
- Suppressions : SupprimerSerie / Saison / Episode

Exemple · Singleton + Factory

DaoFactory.java

```java
public class DaoFactory {
    private static DaoFactory instance;
    private ISerieDao serieDao = null;

    public static final DaoFactory getInstance() throws DaoException {
        if (instance == null) {
            instance = new DaoFactory();
        }
        return instance;
    }
    private DaoFactory() throws DaoException { }

    public ISerieDao getSerieDao() throws DaoException {
        if (this.serieDao == null) {
            this.serieDao = new SerieDaoBdd();
        }
        return this.serieDao;
    }
}
```

Exemple · servlet de liste

ListeSeries.java

```java
@WebServlet(urlPatterns = "/liste")
public class ListeSeries extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        ServletUtils.consumeFlash(request);
        String q = request.getParameter("q");
        boolean onlyAVoir = "1".equals(request.getParameter("aVoir"));
        List<Serie> series = DaoFactory.getInstance()
                .getSerieDao().findSeries(q, onlyAVoir);
        request.setAttribute("series", series);
        getServletContext().getRequestDispatcher("/ListeSeries.jsp")
                .forward(request, response);
    }
}
```

2. Modifications vs PDF

Le sujet cite souvent javax.*, PostgreSQL et un EntityManager static. Ici : Tomcat 10 + Jakarta + MySQL + EM par requête.

- javax.* · jakarta.* (Tomcat 10)
- PostgreSQL d·exemple · MySQL WAMP
- User Library EclipseLink 2.5 · Maven EclipseLink 4
- Generate Tables · ddl-generation create-or-extend-tables
- EM static (p.41) · EMF + 1 EM / requête (thread-safe)
- EAGER (p.52) · LAZY + JOIN FETCH
- Redirect absolu · getContextPath() + "/liste"
- Erreur request + redirect · flash session

Exemple · config JPA adaptée (MySQL / Jakarta)

persistence.xml

```
<persistence-unit name="GestionSeries" transaction-type="RESOURCE_LOCAL">
  <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
  <class>fr.pau.univ.series.model.Serie</class>
  <class>fr.pau.univ.series.model.Saison</class>
  <class>fr.pau.univ.series.model.Episode</class>
  <properties>
    <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
    <property name="jakarta.persistence.jdbc.url"
      value="jdbc:mysql://localhost:3306/GestionSeries?serverTimezone=UTC"/>
    <property name="jakarta.persistence.jdbc.user" value="root"/>
    <property name="eclipselink.ddl-generation" value="create-or-extend-tables"/>
  </properties>
</persistence-unit>
```

Exemple · EntityManager par requête (écart vs PDF p.41)

EntityManagerFilter.java

```
@WebFilter("/*")
public class EntityManagerFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        EntityManager em = SeriesContextListener.getEntityManagerFactory()
                .createEntityManager();
        SeriesContextListener.bind(em);
        try { chain.doFilter(req, res); }
        finally {
            if (em.isOpen()) em.close();
            SeriesContextListener.clear();
        }
    }
}
```

3. Ajouts hors sujet (bonus)

- UI moderne + toasts flash succès/erreur
- ServletUtils (redirect, flash, parseId, rollback)
- DataSeeder si BDD vide
- Recherche q + filtre aVoir + progression %
- Marquer saison vue + ancre/scroll après « vu »
- Indexes SQL fk_serie / fk_saison / nom

Exemple · flash + redirect relatif

ServletUtils.java

```java
public static void setFlashSuccess(HttpServletRequest request, String message) {
    request.getSession(true).setAttribute(FLASH_SUCCESS, message);
}
public static void redirectListe(HttpServletRequest request, HttpServletResponse respon·
        throws IOException {
    response.sendRedirect(request.getContextPath() + "/liste");
}
```

Exemple · bouton ·il (bascule « vu »)

ModifierEtat.java

```java
@WebServlet(urlPatterns = "/etat")
public class ModifierEtat extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        Integer episodeId = ServletUtils.parseId(request.getParameter("episode"));
        String anchor = "ep-" + episodeId;
        Episode e = DaoFactory.getInstance().getEpisodeDao().readEpisode(episodeId);
        e.setVu(!e.isVu());
        DaoFactory.getInstance().getEpisodeDao().updateEpisode(e, true);
        ServletUtils.setFlashSuccess(request,
            e.isVu() ? "Épisode marqué comme vu." : "Épisode marqué à voir");
        ServletUtils.redirectListe(request, response, anchor);
    }
}
```

Exemple · LAZY + JOIN FETCH

Serie.java

```java
@NamedQuery(name = "Serie.findAll",
    query = "SELECT DISTINCT s FROM Serie s "
          + "LEFT JOIN FETCH s.saisons sa LEFT JOIN FETCH sa.episodes")

@OneToMany(fetch = FetchType.LAZY, cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "fk_serie", referencedColumnName = "id")
public List<Saison> getSaisons() { return this.saisons; }
```

Oral & lancement

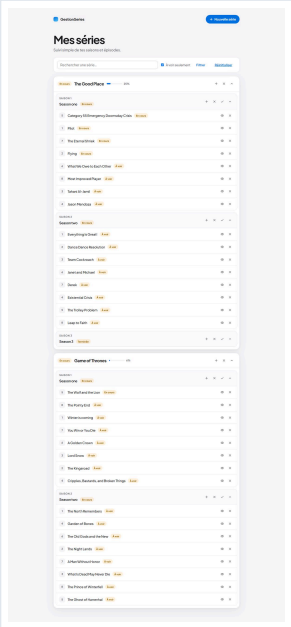
3 arguments solides

- DAO + Factory = découplage métier / stockage
- JPA = entités, JPQL, transactions
- Écarts justifiés : EM/requête, LAZY+JOIN FETCH, flash session

Lancer le projet

- Créer le schéma MySQL GestionSeries
- Vérifier user/mdp dans persistence.xml
- Eclipse · Run on Server (Tomcat 10)
- Ouvrir http://localhost:8080/GestionSeries/

Capture · filtre « À voir seulement »



Filtre activé (case cochée + Réinitialiser).

Version détaillée : README.md à la racine du projet.